[itemize]leftmargin=1.2em,itemsep=2pt,topsep=3pt,parsep=0pt

AI Coding in Finance Fall 2026

Final Project: Design a Long-Running Agent Workflow for a Finance Task

Individual project. Choose a substantial research or analytical activity and design an end-to-end agent workflow for it. The scope should justify multiple linked stages, repeated evaluation, persistent state, and recovery—not merely a single prompt or report update.

1. Specify a substantial, recurring task

Choose an activity from your work or study. For example, **buy-side quantitative strategy research**, spanning hypotheses, data preparation, backtesting, robustness checks, portfolio construction, and ongoing monitoring; or **sell-side company/industry coverage**, spanning source collection, financial modeling, valuation, report production, and continuing updates. Other tasks, even those weakly related to finance, are welcome if comparable in scope. Define the intended user, current process, stages and dependencies, recurring triggers, inputs, permitted sources, information cutoffs, work products, and acceptance criteria. Explain why this needs a persistent workflow rather than a one-shot script.

2. Design the architecture using the lecture concepts

Provide an annotated workflow diagram and a short design note explaining:

- **Roles and handoffs:** agent responsibilities, tools, reusable skills or instructions, and input/output artifacts. Justify the division of work; use deterministic code for calculations where appropriate. Distinguish agent decisions from the harness/controller that manages execution.
- **Long-running operation:** persistent state, checkpoints, scheduling or new-input triggers, dependencies, retries, and stop/escalation rules. Explain how an interrupted run resumes without repeating completed actions, and set limits on time, cost, and tool permissions.
- **Verification and improvement:** checks independent of the producing agent, human approval before consequential actions, and a bounded revision loop. Explain how a proposed change is tested against a fixed baseline, accepted or rejected, versioned, and rolled back. For time-sensitive tasks, prevent future-information leakage.

3. Submit one ZIP with complete Git history

Include your design note and diagram, prompts/skills/configuration, code if implemented, shareable example inputs and outputs, and verification/recovery evidence. The root `README.md` must give your identification, task summary, **repository URL** (if applicable), file guide, and setup/run or walkthrough instructions (including dependencies where relevant). Disclose concisely what you input to the project. Use meaningful commits throughout development; do not submit only a final-file dump.

Submit one ZIP of your **existing local project folder**, including all deliverables and its **complete hidden `.git/` directory**. Commit final changes first, and ensure the repository has full, non-shallow history. Include hidden folders when compressing. Name the ZIP file as `studentID-final-project.zip`.

A host's "Download ZIP" or `git archive` is not sufficient: neither includes Git history. Files and history must be reviewable without repository access. Never include credentials, private client data, or secrets in files *or commit history*.